

Trí Tuệ Nhân Tạo

(Artificial Intelligence)

Lê Thanh Hương

Viện Công nghệ thông tin và Truyền thông
Trường Đại Học Bách Khoa Hà Nội
Năm 2020

Nội dung môn học

- Chương 1. Tổng quan
- Chương 2. Tác tử thông minh
- Chương 3. Giải quyết vấn đề
- Chương 4. Tri thức và suy diễn
- **Chương 5. Biểu diễn tri thức**
- Chương 6. Học máy
 - Giới thiệu về học máy
 - K láng giềng gần
 - Phân lớp Naïve Bayes
 - Học cây quyết định
 - Mạng nơron

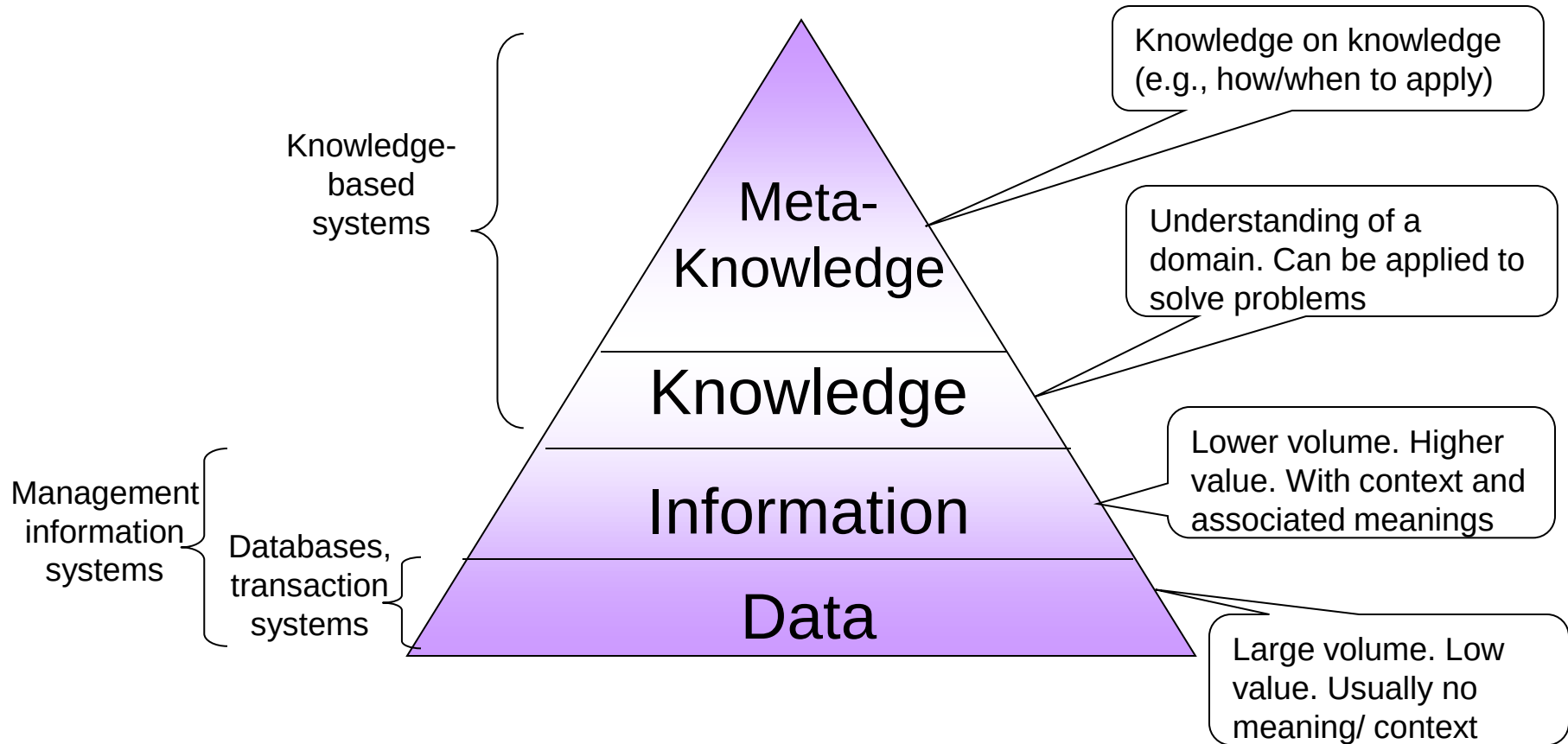
Dữ liệu, Thông tin, Tri thức (1)

- **Dữ liệu (data)** thường được định nghĩa là các sự kiện (facts) hoặc các ký hiệu (symbols)
- **Thông tin (information)** thường được định nghĩa là dữ liệu đã được xử lý hoặc chuyển đổi thành những dạng hoặc cấu trúc phù hợp cho việc sử dụng của con người
 - Thu được sau khi loại bỏ những thứ dư thừa và giữ lại phần cốt lõi.
- **Tri thức (knowledge)** thường được định nghĩa là sự hiểu biết (nhận thức) về thông tin
 - Thu được sau quá trình nhận thức, phát hiện, hoặc học tập

Dữ liệu, Thông tin, Tri thức (2)

- Dữ liệu
 - Nhiệt độ ngoài trời là 5 độ C
- Thông tin
 - Ngoài trời thời tiết lạnh
- Tri thức
 - Nếu ngoài trời thời tiết lạnh thì bạn nên mặc áo choàng ấm (khi đi ra ngoài)

Dữ liệu, Thông tin, Tri thức (3)



(Adapted from “Knowledge Engineering course (CM3016), by K. Hui 2008-2009”)

Biểu diễn tri thức

- Biểu diễn tri thức (Knowledge representation) là một bài toán quan trọng của Trí tuệ nhân tạo
 - các phương pháp, cách thức biểu diễn tri thức và các công cụ hỗ trợ việc biểu diễn tri thức
- Tồn tại nhiều phương pháp biểu diễn tri thức
 - Luật sản xuất (Production rules)
 - Khung (Frames)
 - Mạng ngữ nghĩa (Semantic networks)
 - Ontology
 - Các mô hình xác suất (probabilistic models)
 - Học sâu (deep learning)
 - ...

Biểu diễn tri thức: vấn đề

- Tính hoàn chỉnh (Completeness)
 - Phương pháp biểu diễn có hỗ trợ việc thu thập và thể hiện **mọi khía cạnh** của tri thức (của một lĩnh vực cụ thể)?
- Tính ngắn gọn (Conciseness)
 - Phương pháp biểu diễn có cho phép việc **thu thập tri thức một cách hiệu quả**?
 - Phương pháp biểu diễn có cho phép việc **lưu trữ và truy nhập dễ dàng** tri thức không?
- Tính hiệu quả về tính toán (Computational efficiency)
- Tính rõ ràng, dễ hiểu (Transparency)
 - Phương pháp biểu diễn có cho phép diễn giải (để **người dùng hiểu**) về các hoạt động và các kết luận của hệ thống?

Biểu diễn tri thức: bằng luật (1)

- Biểu diễn tri thức bằng các luật (rules) là cách biểu diễn phổ biến trong các hệ cơ sở tri thức
 - Một luật chứa đựng (biểu diễn) tri thức về việc giải quyết một vấn đề nào đó
 - Các luật được tạo nên khá dễ dàng, và dễ hiểu
- Một luật được biểu diễn ở dạng:

nếu...thì...

IF A_1 AND A_2 AND ... AND A_n THEN B

- **A_i**
 - Là các **điều kiện (conditions, antecedents, premises)**
- **B**
 - Là **kết luận (conclusion, consequence, action)**

Biểu diễn tri thức: bằng luật (2)

■ Mệnh đề điều kiện của một luật

- Không cần sử dụng toán tử logic OR
- Một luật với toán tử logic OR trong mệnh đề điều kiện, thì sẽ được chuyển thành một tập các luật tương ứng không chứa OR
- Ví dụ: Luật (IF $A_1 \vee A_2$ THEN B) được chuyển thành 2 luật (IF A_1 THEN B) và (IF A_2 THEN B)

$$\begin{aligned}(A_1 \vee A_2) \Rightarrow B & \Leftrightarrow \overline{A_1 \vee A_2} \vee B \\ & \Leftrightarrow (\overline{A_1} \wedge \overline{A_2}) \vee B \\ & \Leftrightarrow (\overline{A_1} \vee B) \wedge (\overline{A_2} \vee B) \\ & \Leftrightarrow (A_1 \Rightarrow B) \wedge (A_2 \Rightarrow B)\end{aligned}$$

■ Mệnh đề kết luận của một luật

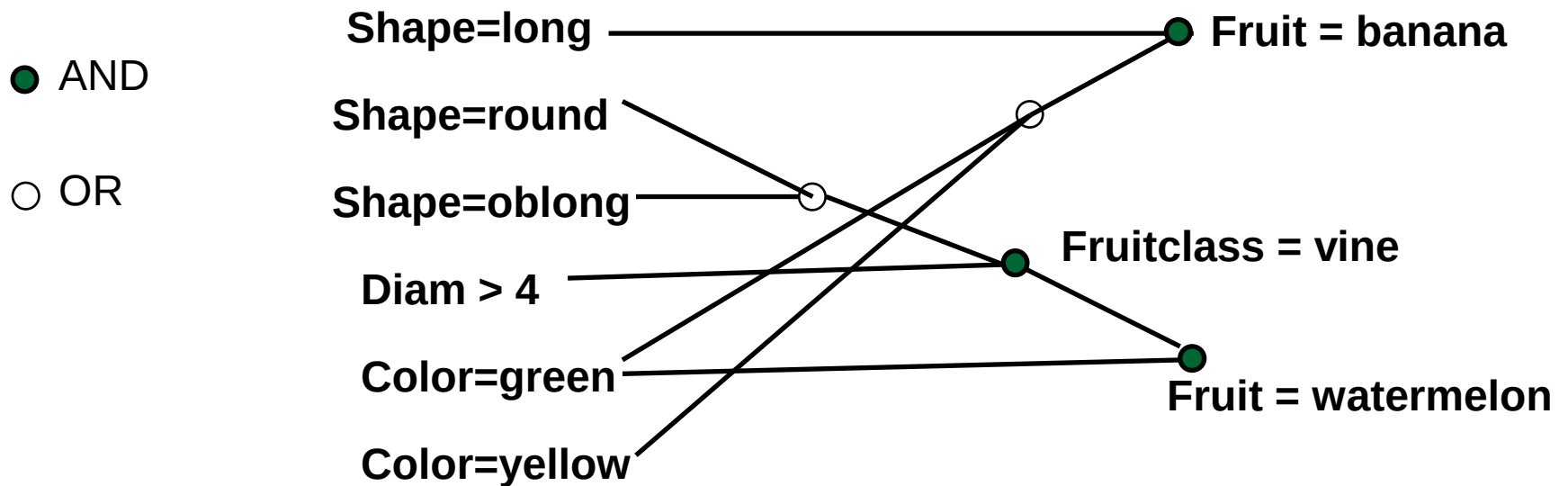
- Không cần sử dụng toán tử logic AND
- Một luật với toán tử logic AND trong mệnh đề kết luận, thì sẽ được chuyển thành một tập các luật tương ứng không chứa AND
- Ví dụ: Luật (IF ... THEN $B_1 \wedge B_2$) được chuyển thành 2 luật (IF ... THEN B_1) and (IF ... THEN B_2)
- **Không cho phép sử dụng toán tử OR!**

Các kiểu luật

- Các kiểu luật khác nhau để biểu diễn các kiểu tri thức khác nhau
- Quan hệ liên kết
 - `IF addressAt(x, Hospital) THEN heathIs(x, Bad)`
- Quan hệ nguyên nhân (kết quả)
 - `IF diseaseType(x, Infection) THEN tempIs(x, High)`
- Tình huống và hành động (gợi ý)
 - `IF diseaseType(x, Infection) THEN
takeMedicine(x, Antibiotic)`
- Quan hệ logic
 - `IF tempGreater(x, 37) THEN isFever(x)`

Đồ thị AND/OR (1)

- IF (Shape=long) **AND** (Color=(green **OR** yellow)) THEN (Fruit=banana)
- IF (Shape=(round **OR** oblong)) **AND** (Diam > 4) THEN (Fruitclass=vine)
- IF (Fruitclass=vine) **AND** (Color=green) THEN (Fruit=watermelon)



Đồ thị AND/OR (2)

- Luật *IF (Shape=long) **AND** (Color=(green **OR** yellow)) THEN (Fruit=banana)* được tạo nên bởi các luật:
 - IF (Shape=long) **AND** (Color=green) THEN (Fruit=banana)
 - IF (Shape=long) **AND** (Color=yellow) THEN (Fruit=banana)
- Luật *IF (Shape=(round **OR** oblong)) **AND** (Diam > 4) THEN (Fruitclass=vine)* được tạo nên bởi các luật:
 - IF (Shape=round) **AND** (Diam > 4) THEN (Fruitclass=vine)
 - IF (Shape=oblong) **AND** (Diam > 4) THEN (Fruitclass=vine)

Vài vấn đề với biểu diễn luật

- Các luật có chứa các vòng lặp
 - IF A THEN A
 - {IF A THEN B, IF B THEN C, IF C THEN A}
- Các luật có chứa mâu thuẫn
 - {IF A THEN B, IF B THEN C, IF A AND D THEN $\neg$ C}
- Các kết luận không thể suy ra được (từ các luật hiện có)
- Khó khăn trong việc thay đổi (cập nhật) cơ sở tri thức
 - Cơ sở tri thức cũ: {IF A_1 THEN B_1 , IF A_2 THEN B_2 , ..., IF A_n THEN B_n }
 - Cần bổ sung thêm điều kiện C vào tất cả các luật
 - Cơ sở tri thức mới: {IF A_1 AND C THEN B_1 , IF A_2 AND C THEN B_2 , ..., IF A_n AND C THEN B_n }

Sử dụng các luật trong suy diễn

- So khớp mẫu (Pattern matching)
 - Để kiểm tra một luật có thể được sử dụng (áp dụng) hay không
 - Ví dụ: Nếu cơ sở tri thức chứa đựng tập các luật $\{ \text{IF } A_1 \text{ THEN } B_1, \text{ IF } A_1 \text{ AND } A_2 \text{ THEN } B_2, \text{ IF } A_2 \text{ AND } A_3 \text{ THEN } B_3 \}$ và các sự kiện (được lưu trong bộ nhớ làm việc) bao gồm A_1 và A_2 , thì 2 luật đầu tiên có thể được sử dụng
- Chuỗi suy diễn (chuỗi áp dụng các luật)
 - Xác định trật tự áp dụng các luật trong quá trình suy diễn
 - Với một tập các luật và một tập các sự kiện (các giả thiết), các luật nào nên được sử dụng, và theo trật tự nào, để đạt tới (suy ra) một kết luận cần chứng minh?
 - 2 chiến lược suy diễn: tiến (forward) vs. lùi (backward)
 - 2 chiến lược suy diễn này đã được trình bày trong bài trước!

Giải quyết xung đột

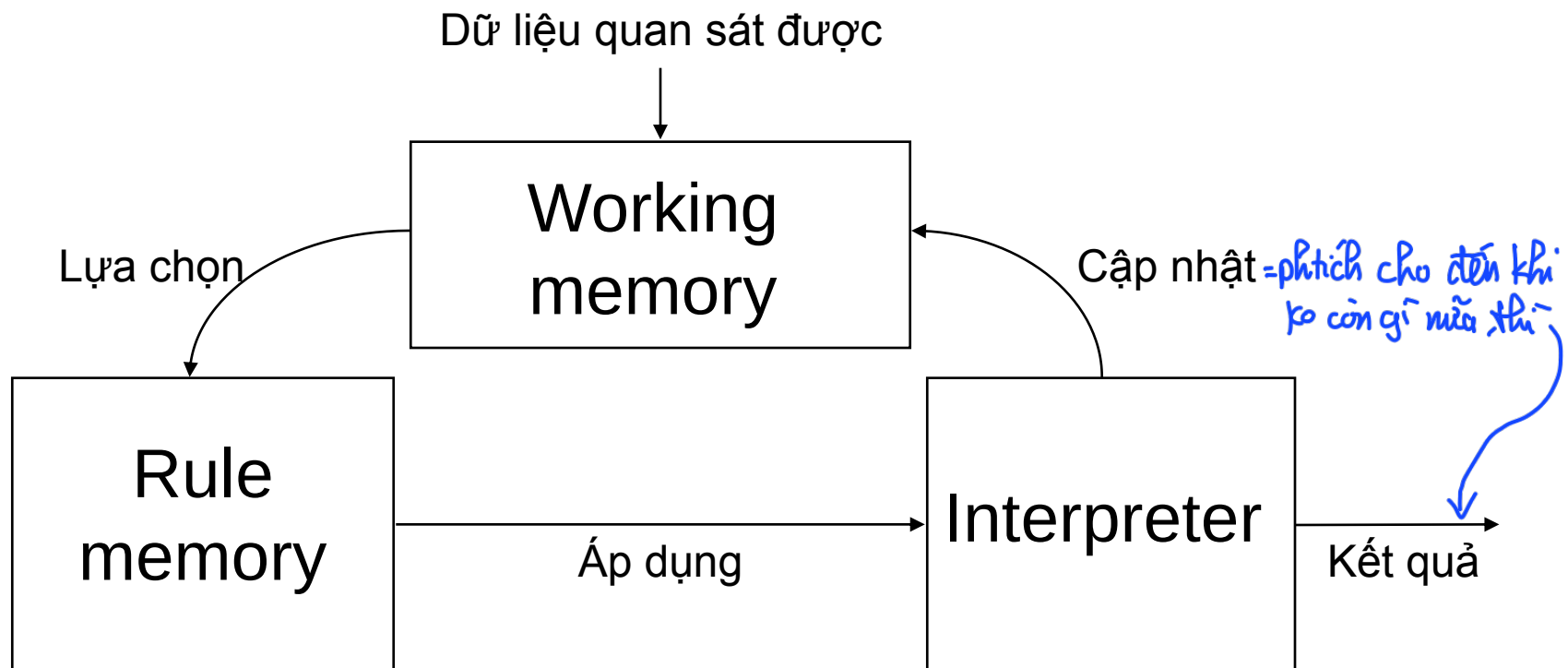
- Một xung đột (conflict) xảy ra khi có nhiều hơn một luật có thể áp dụng được (phù hợp với các sự kiện trong bộ nhớ làm việc)
 - Lưu ý, một xung đột không phải là một mâu thuẫn của tập luật
- Trong trường hợp xảy ra xung đột, cần một chiến lược giải quyết xung đột (conflict resolution strategy - CRS) để quyết định luật nào được (ưu tiên) áp dụng
- Sự lựa chọn thích hợp một chiến lược giải quyết xung đột có thể mang lại những cải thiện đáng kể đối với quá trình suy diễn của hệ thống

Chiến lược giải quyết xung đột

- Áp dụng luật **xuất hiện đầu tiên** (theo thứ tự) trong cơ sở tri thức
- Không áp dụng các luật sinh ra các kết quả (sự kiện) đã có trong bộ nhớ làm việc
- Áp dụng **luật cụ thể nhất** (luật có nhiều điều kiện nhất)
- Áp dụng các luật phù hợp với các sự kiện được đưa vào trong bộ nhớ làm việc **gần thời điểm hiện tại nhất**
- Không áp dụng lại một luật, nếu nó vẫn sinh ra cùng một tập các sự kiện (giống như lần áp dụng trước của nó)
- Áp dụng **luật có độ tin cậy** (chắc chắn) cao nhất
- ...
- *Kết hợp* của các chiến lược trên

Hệ thống suy diễn dựa trên luật (1)

Kiến trúc điển hình của một hệ thống suy diễn dựa trên luật (Rule-based system – RBS)



(<http://www.cwa.mdx.ac.uk/bis2040/johnlect.html>)

Hệ thống suy diễn dựa trên luật (2)

■ Bộ nhớ làm việc (Working memory)

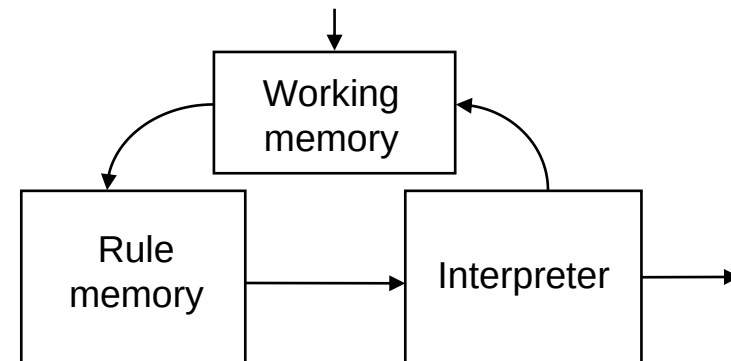
- Lưu giữ các sự kiện (các giả thiết đúng, đã được chứng minh)
- Các sự kiện này sẽ quyết định những luật nào được áp dụng (bởi thành phần Interpreter)

■ Bộ nhớ các luật (Rule memory)

- Chính là cơ sở tri thức của hệ thống
- Lưu giữ các luật có thể áp dụng

■ Bộ diễn dịch (Interpreter)

- Hệ thống bắt đầu bằng việc đưa một sự kiện (dữ liệu) phù hợp vào bộ nhớ làm việc
- Khi sự kiện (dữ liệu) trong bộ nhớ làm việc phù hợp với các điều kiện của một luật trong bộ nhớ các luật, thì luật đó sẽ được áp dụng



RBS: Ưu điểm (1)

- Cách biểu diễn (diễn đạt) phù hợp
 - Rất gần với cách diễn đạt trong ngôn ngữ tự nhiên
 - Rất dễ dàng để diễn đạt nhiều tri thức bởi các luật
- Dễ hiểu
 - Các luật dạng IF-THEN rất dễ hiểu đối với người sử dụng
 - Trong một lĩnh vực (bài toán) cụ thể, cách biểu diễn bằng luật giúp các chuyên gia trong lĩnh vực này có thể đánh giá và cải tiến các luật

RBS: Ưu điểm (2)

- Một cách biểu diễn tri thức theo kiểu khai báo (declarative)
 - Thu thập các tri thức (ở dạng các luật IF-THEN) về một lĩnh vực cụ thể, và đưa chúng vào trong một cơ sở các luật (rule base)
 - (Có thể) không cần phải quan tâm đến khi nào, làm thế nào, và theo trật tự nào mà các luật được sử dụng – Hệ thống sẽ tự động đảm nhận các nhiệm vụ này

RBS: Nhược điểm

- Khả năng biểu diễn (diễn đạt) bị giới hạn
 - Trong nhiều lĩnh vực bài toán thực tế, tri thức của lĩnh vực bài toán đó không phù hợp với cách biểu diễn dạng (IF-THEN)
- Sự tương tác giữa các luật và trật tự của các luật trong cơ sở luật có thể gây ra các hiệu ứng không mong muốn
 - Trong quá trình thiết kế (design) và bảo trì (maintenance) một cơ sở luật, mỗi luật mới được đưa vào cần phải được cân nhắc (kiểm tra) với các luật đã có từ trước
 - Rất khó khăn và chi phí tốn kém để xem xét tất cả các tương tác (interactions) có thể giữa các luật
- Quá tốn kém chi phí: thời gian, công sức, tiền bạc

Biểu diễn tri thức: bảng khung (1)

- Làm thế nào để biểu diễn tri thức “Xe buýt màu blue”?
- Giải pháp thứ 1. `Blue (bus)`
 - Câu hỏi “Cái gì là màu blue?” có thể trả lời được = *Blue(x)*
 - Nhưng câu hỏi “Màu của xe buýt là gì?” thì không thể trả lời được
Blue = tên của logic vị từ, k'p' tham số của mệnh đề logic
- Giải pháp thứ 2. `Color (bus, blue)`
 - Câu hỏi “Cái gì có màu blue?” có thể trả lời được
 - Câu hỏi “Màu của xe buýt là gì?” có thể trả lời được
 - Nhưng câu hỏi “Thuộc tính nào của xe buýt có giá trị là màu blue?” thì không thể trả lời được
- Giải pháp thứ 3. `Prop (bus, color, blue)`
 - Tất cả 3 câu hỏi trên đều có thể trả lời được

Biểu diễn tri thức: bảng khung (2)

- Một đối tượng được biểu diễn bởi:
(Object, Property, Value)
 - Được gọi là cách biểu diễn bằng bộ ba
đối tượng-thuộc tính-giá trị (object-property-value)
- Nếu chúng ta gộp nhiều thuộc tính của cùng một kiểu đối tượng thành một cấu trúc, thì chúng ta có cách biểu diễn hướng đối tượng (object-centered representation)

$Prop(Object, Property_1, Value_1)$

$Prop(Object, Property_2, Value_2)$

...

$Prop(Object, Property_n, Value_n)$

Object

Property₁

Property₂

...

Property_n

Biểu diễn hướng đối tượng

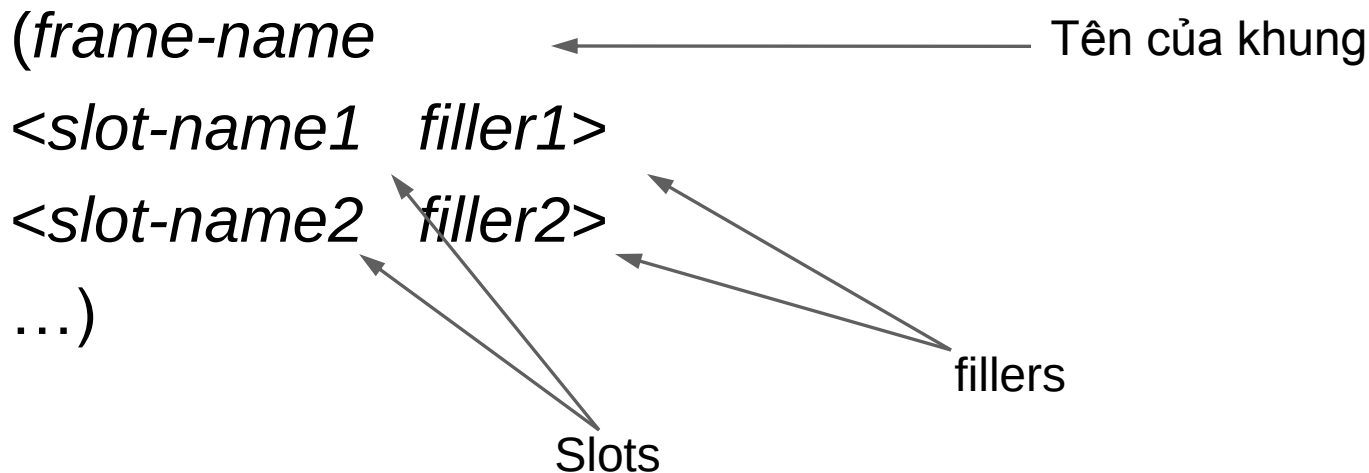
- Cách biểu diễn bằng bộ ba *object-property-value* là một cách tự nhiên để biểu diễn các đối tượng
- Các đối tượng cụ thể (Physical objects)
 - Ví dụ: Một *cái bàn* có các thuộc tính chất liệu bề mặt, số ngăn kéo, độ rộng, độ dài, độ cao, màu sắc, ...
- Các tình huống (Situations)
 - Ví dụ: Một *lớp học* có các thuộc tính mã số phòng học, danh sách sinh viên tham dự, giáo viên, ngày học, thời gian học, ...
 - Ví dụ: Một *chuyến đi nghỉ mát* có các thuộc tính nơi khởi hành, nơi đến, phương tiện di chuyển, phòng nghỉ, ...

Khung (Frame)

- Có 2 kiểu khung: cụ thể (individual) và tổng quát (generic)
- **Khung cụ thể (Individual frames)**. Để biểu diễn một đối tượng cụ thể, chẳng hạn như một người cụ thể, một chuyến đi nghỉ mát cụ thể, ...
khai báo trừu tượng của khung cụ thể
- **Khung tổng quát (Generic frames)**. Để biểu diễn một lớp (loại) các đối tượng, chẳng hạn như các sinh viên, các chuyến đi nghỉ mát, ...
- Ví dụ
 - Khung tổng quát: European_City
 - Khung cụ thể: City_Paris

Biểu diễn của một khung

- Một khung được biểu diễn bằng một danh sách định danh các thuộc tính được gọi là các **slots**
- Giá trị gán cho một thuộc tính được gọi là **filler of the slot**



Khung đơn *khung cụ thể*

- Một khung đơn (individual frame) có một thuộc tính đặc biệt có tên là **INSTANCE-OF**, và giá trị của thuộc tính đặc biệt này là tên của một khung tổng quát (generic frame) ✓
- Ví dụ
(toronto % sử dụng chữ thường cho các khung đơn
<:**INSTANCE-OF** CanadianCity>
<:Province ontario>
<:Population 4.5M>
...)

Khung tổng quát

- Một khung tổng quát (generic frames) có thể có một thuộc tính đặc biệt là **IS-A** mà giá trị của thuộc tính đặc biệt này là tên của một khung tổng quát khác
- Ví dụ

(CanadianCity % sử dụng tên bắt đầu chữ hoa cho khung tổng quát
<:**IS-A** City>
<:Province CanadianProvince>
<:Country canada>
...)

Suy diễn với khung (1) ~ method loop

- Các thuộc tính (slots) trong các khung tổng quát có thể được gán (liên kết) với các **thủ tục** để thực hiện và điều khiển việc suy diễn
- Có 2 kiểu thủ tục: **IF-NEEDED** và **IF-ADDED** *gán giá trị cho các biến trg TH nó đc khởi tạo / mới đc thêm vào*
- Thủ tục **IF-NEEDED**
 - Được thực hiện khi không có giá trị cần thiết được gán cho một thuộc tính (no slot filler)?
 - Ví dụ: (Table
 <:Clearance [**IF-NEEDED** computeClearance]>
 ...)

computeClearance là một thủ tục để tính toán (xác định) mức độ sạch sẽ của cái bàn

Suy diễn với khung (2)

■ Thủ tục **IF-ADDED**

- Được thực hiện khi một thuộc tính được gán giá trị, để cho phép lan truyền ảnh hưởng của việc gán giá trị của thuộc tính đó đối với các khung khác (ví dụ, để đảm bảo các ràng buộc trong bài toán)

- Ví dụ: (Lecture

<:DayOfWeek WeekDay>

<:Date [**IF-ADDED** computeDayOfWeek]>

...)

Giá trị của thuộc tính :DayOfWeek sẽ được tính toán (lại) khi thuộc tính :Date được gán giá trị

Khung: Giá trị mặc định (1)

- Hãy xét khung tổng quát (generic frame) sau đây

(CanadianCity

<:**IS-A** City>

<:Province CanadianProvince>

<:Country canada>

...)

- Hãy xét khung cụ thể (individual frame) sau đây

(city134

<:**INSTANCE-OF** CanadianCity>

...)

- Đối với khung `city134`, giá trị (mặc định) cho thuộc tính *:Country* là *canada*

Khung: Giá trị mặc định (2)

- Hãy xét khung cụ thể (individual frame) sau đây
(city135
<:**INSTANCE-OF** CanadianCity>
<:Country holland>
...)
- Đối với khung `city135`, thì giá trị cho thuộc tính
:*Country* là *holland* (chứ không phải là giá trị mặc định
canada) đã ghi đè

Khung: Tính kế thừa (1)

- Các thủ tục và các giá trị thuộc tính của một khung tổng quát hơn sẽ được áp dụng (kế thừa) bởi một khung cụ thể hơn, thông qua cơ chế kế thừa

- Ví dụ

(CoffeeTable
<:**IS-A** Table>
...)

(MahoganyCoffeeTable
<:**IS-A** CoffeeTable>
...)

Khung: Tính kế thừa (2)

■ Ví dụ

(Elephant

<:**IS-A** Mammal>

<:Colour gray>

...)

(RoyalElephant

<:**IS-A** Elephant>

<:Colour white>

...)

(clyde

<:INSTANCE-OF RoyalElephant>

...)

Khung: Suy diễn

- Quá trình suy diễn trong phương pháp biểu diễn bằng khung sẽ diễn ra như sau
 1. Người dùng khởi tạo một khung (tương đương với việc khai báo sự tồn tại của một đối tượng hay một tình huống)
 2. Các giá trị của các thuộc tính sẽ được kế thừa (từ các khung tổng quát hơn)
 3. Các thủ tục IF-ADDED sẽ được thực hiện. Việc này có thể sẽ dẫn đến việc khởi tạo của các khung khác, và việc gán giá trị của các thuộc tính
- Nếu người dùng hoặc một thủ tục yêu cầu việc gán giá trị cho một thuộc tính, thì:
 - Nếu có giá trị cho thuộc tính, thì giá trị đó sẽ được gán
 - Nếu không, thủ tục IF-NEEDED sẽ được thực hiện

Biểu diễn bằng khung: Ưu điểm

- Kết hợp được cả tri thức khai báo (declarative knowledge) và tri thức thủ tục (procedural knowledge) trong cùng một phương pháp biểu diễn
- Các khung được tổ chức có cấu trúc phân cấp, cho phép dễ dàng phân loại (phân lớp) tri thức
- Cấu trúc phân cấp các khung cho phép giảm bớt sự phức tạp (và chi phí) trong quá trình xây dựng cơ sở tri thức
- Cho phép thiết lập các ràng buộc đối với các giá trị được gán cho các thuộc tính (ví dụ: ràng buộc giá trị nhập vào phải nằm trong một khoảng giá trị cụ thể)
- Cho phép lưu giữ các giá trị mặc định (sử dụng thuộc tính đặc biệt IS-A, các giá trị của các thuộc tính của một khung tổng quát hơn được sử dụng để gán cho các thuộc tính của một khung cụ thể hơn)

Biểu diễn bằng khung: Nhược điểm

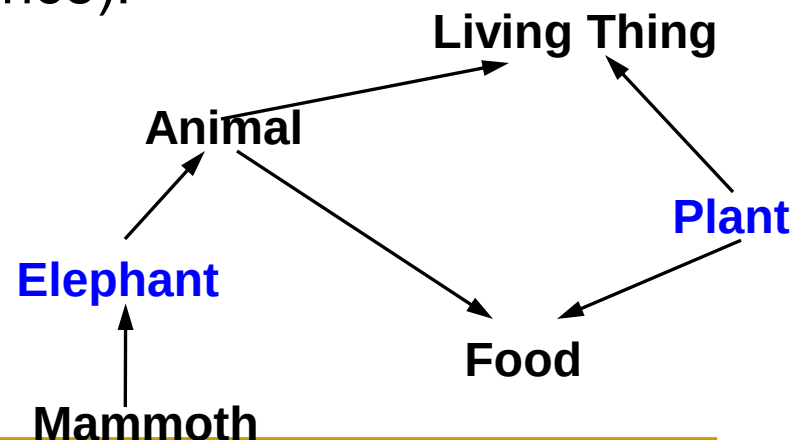
- Trong quá trình thiết kế cấu trúc phân cấp của các khung, cần rất để ý đến sự hợp lý của việc phân loại (các khung)
- Có thể gặp vấn đề chi phí cao cho việc thiết kế các thủ tục (IF-ADDED và IF-NEEDED) – Quá nhiều công sức dành cho việc thiết kế các thủ tục phù hợp, thay vì tập trung vào việc kiểm tra cấu trúc và nội dung của các khung
- Quá trình khai thác các khung có thể không hiệu quả, vì không có phương pháp hiệu quả để lưu trữ dữ liệu (của các khung) trong máy tính

Mạng ngữ nghĩa (1)

- **Mạng ngữ nghĩa (Semantic Network)** được đề cử bởi Quillian vào năm 1966 như là một mô hình biểu diễn bộ nhớ của con người
- Các động cơ thúc đẩy sự phát triển của mạng ngữ nghĩa
 - ❑ Để hiểu về cấu trúc của bộ nhớ con người, và việc sử dụng cấu trúc bộ nhớ này trong xử lý (hiểu) ngôn ngữ
 - ❑ Kiểu biểu diễn nào cho phép lưu giữ các ý nghĩa (meanings) của các từ để có thể sử dụng lại các ngữ nghĩa này (như trong bộ nhớ con người)?
 - ❑ Các chứng minh về tâm lý học đã chỉ ra rằng bộ nhớ của con người sử dụng các liên kết trong việc xử lý (hiểu) các từ
- **Yêu cầu:** cần biểu diễn định nghĩa từ điển của các từ, để
 - ❑ So sánh và phân biệt ý nghĩa của 2 từ
 - ❑ Sinh ra các câu “tựa” (gần giống) tiếng Anh để mô tả sự so sánh này

Mạng ngữ nghĩa (2)

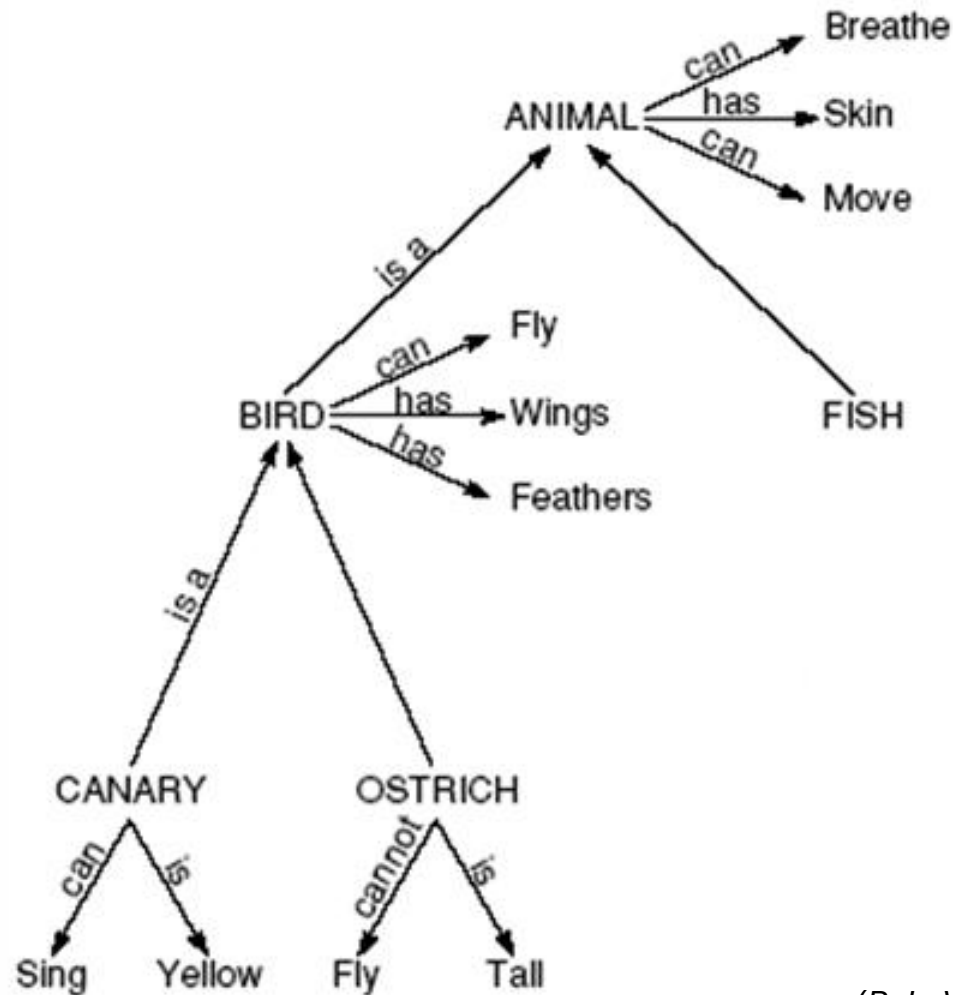
- Mạng ngữ nghĩa (Semantic Network) là phương pháp biểu diễn ngữ nghĩa dựa trên đồ thị (graph-based representation)
- Một mạng ngữ nghĩa bao gồm một tập **các nút (nodes)** và **các liên kết (links)**
 - Các nút biểu diễn các khái niệm
 - Các liên kết biểu diễn các mối quan hệ giữa các khái niệm
- Quá trình suy diễn (reasoning/inference): được thực hiện thông qua cơ chế lan truyền
 - Tác động (Activation)
 - Kế thừa (Inheritance)



Mạng ngữ nghĩa: Cú pháp

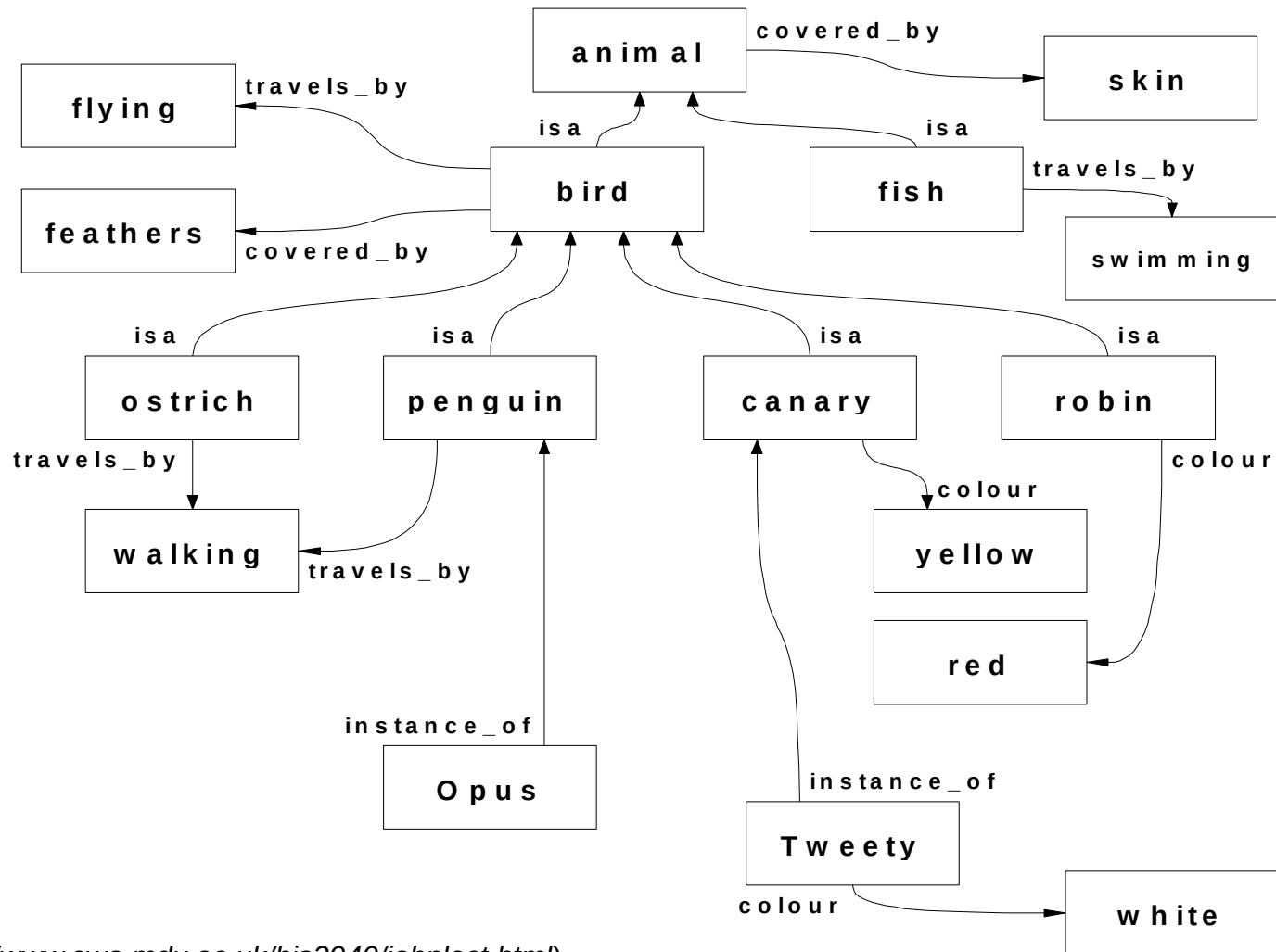
- **Các nút (nodes)** biểu diễn các *khái niệm (concepts)*, *hành động (actions)* hoặc *đối tượng (objects)* trong lĩnh vực bài toán đang xét
- **Các liên kết (links)** là các quan hệ được gán nhãn và có chiều (directional and labeled) giữa các nút
- Hai kiểu liên kết: **kế thừa** và **cụ thể**
- **Liên kết kế thừa (Inheritance-oriented link)** biểu diễn:
 - Nút A là **một lớp (loại) con** của nút B (vd: liên kết *IS-A*)
 - Nút A là **một ví dụ (instance)** của nút B (vd: liên kết *INSTANCE-OF*)
- **Liên kết cụ thể (Domain-specific link)** biểu diễn:
 - Nút A liên quan tới (có quan hệ với) nút B
 - Ví dụ: *HAS, CAN, HAS-PART, CAUSES, HAS-COLOR, ...*

Mạng ngữ nghĩa: Ví dụ (1)



(B. L. Vrusias, course AI-CS289)

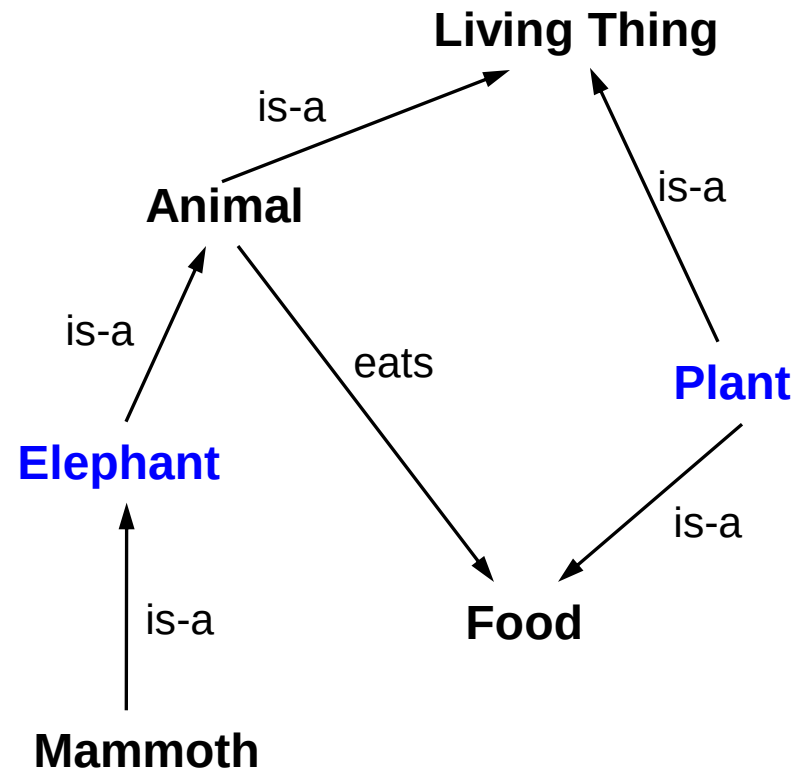
Mạng ngữ nghĩa: Ví dụ (2)



(<http://www.cwa.mdx.ac.uk/bis2040/johnlect.html>)

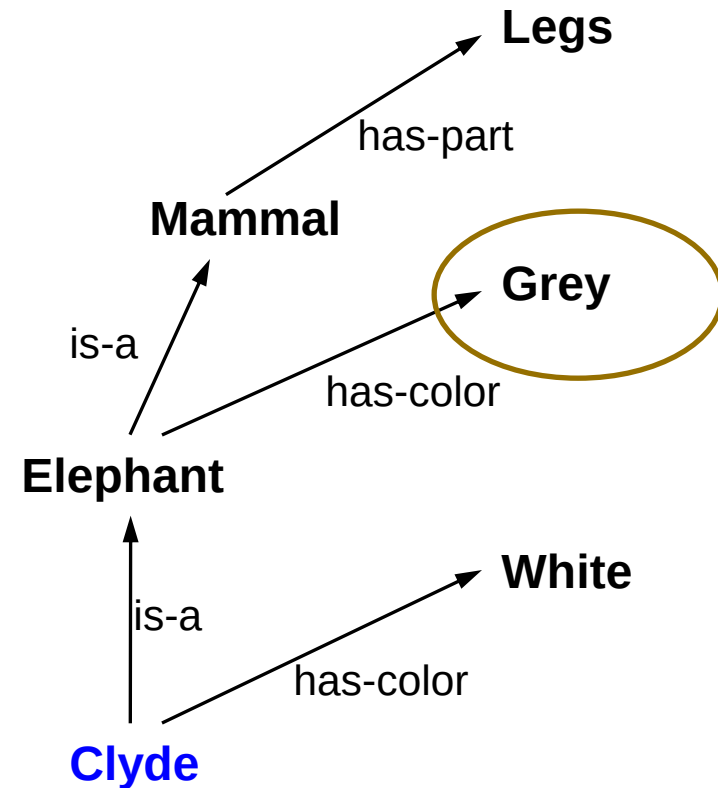
SN: Lan truyền tác động

- Đối với 2 khái niệm, việc lan truyền tác động (spreading activation) sẽ kích hoạt tác động từ khái niệm này tới khái niệm kia, hoặc theo cả 2 hướng
- Cho phép xác định các khái niệm “nằm giữa” liên quan đến cả 2 khái niệm đó
 - Ví dụ: Xét việc lan truyền tác động giữa 2 khái niệm “Elephant” và “Plant”



SN: Tính kế thừa

- Các thuộc tính (properties) của lớp (loại) cha được kế thừa cho các lớp (loại) con
- **Kế thừa toàn bộ (Universal inheritance):** Tất cả các quan hệ được kế thừa
- **Kế thừa mặc định (Default inheritance):** Các quan hệ được kế thừa, trừ khi có các thông tin mâu thuẫn (với nút cha) ở một nút con

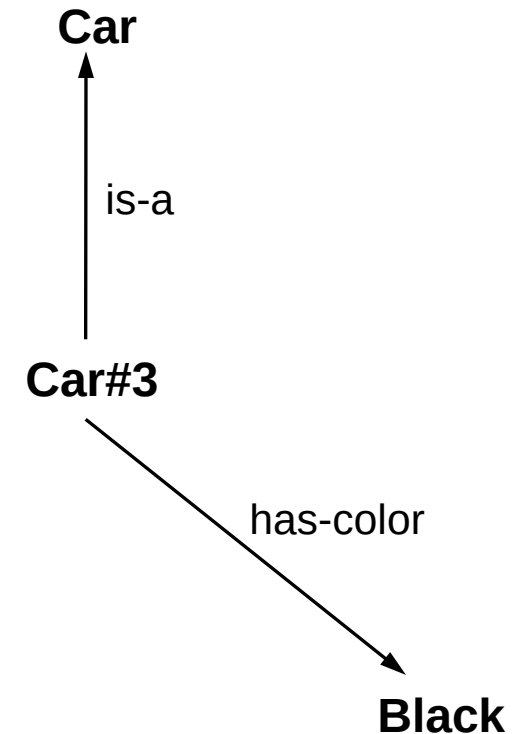


SN: Ngữ nghĩa (1)

- Biểu diễn bằng mạng ngữ nghĩa rất trực quan, và gần với nhận thức (cách biểu diễn) của con người
- Nhưng: Đối với *cùng một đồ thị (mạng) ngữ nghĩa*, các hệ thống khác nhau có thể có *các cách diễn giải (interpretations) khác nhau*
- Ngữ nghĩa (semantics) của các mạng ngữ nghĩa?
 - "Since the semantics of any given language is dependent of the interpretation of the primitive elements....., the well-definedness of a network language rests heavily on the set of node and link types that it provides" (Brachman, p204, Readings in KR)

SN: Ngữ nghĩa (2)

- Ý nghĩa của mạng ngữ nghĩa sau là gì?
 - ❑ Thể hiện định nghĩa của một cái ô-tô màu đen (**Thông tin định nghĩa**)
 - ❑ Thể hiện rằng tồn tại một ô-tô màu đen (**Thông tin xác nhận**)
 - ❑ Thể hiện rằng một cái ô-tô cụ thể (Car#3) là màu đen (**Xác nhận sự tồn tại**)



SN: Ngữ nghĩa (3)

■ Các liên kết có thể là

□ Liên kết xác nhận (Assertional links)

- Lưu giữ **các thông tin về không gian bài toán** đang xét
- *Có thể thay đổi* khi không gian bài toán thay đổi
- Ví dụ: “John hit Mary” (một *sự kiện* cụ thể đã xảy ra)

□ Liên kết định nghĩa (Definitional links)

- Lưu giữ **các ý nghĩa của các khái niệm**
- *Không thay đổi* khi không gian bài toán thay đổi
- Ví dụ: “apple *is-a* fruit”, “apple *has-color* red”

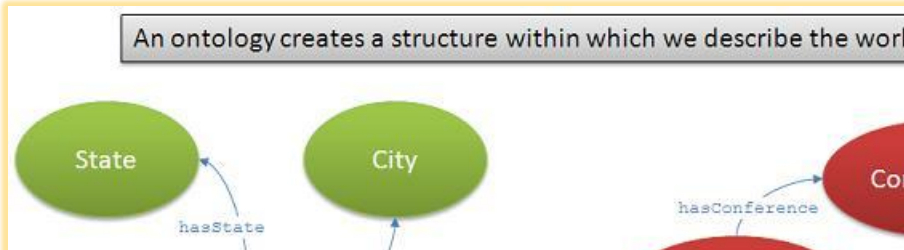
Mạng ngữ nghĩa: Ưu điểm

- Rõ ràng (trực quan) trong hiển thị, dễ hiểu đối với người dùng
 - SNs thường được sử dụng như là một công cụ trao đổi (làm việc) giữa các kỹ sư tri thức (knowledge engineers) và các chuyên gia (experts) trong giai đoạn thu thập tri thức
- SNs là rất phù hợp đối với các bài toán biểu diễn tri thức ở dạng phân cấp các khái niệm
 - Tri thức được phân loại (phân lớp) thành một cấu trúc phân cấp
- Cơ chế biểu diễn phân cấp của SNs hỗ trợ quá trình suy diễn nhanh chóng
- Tập trung vào các thành phần chính của tri thức và liên kết giữa chúng

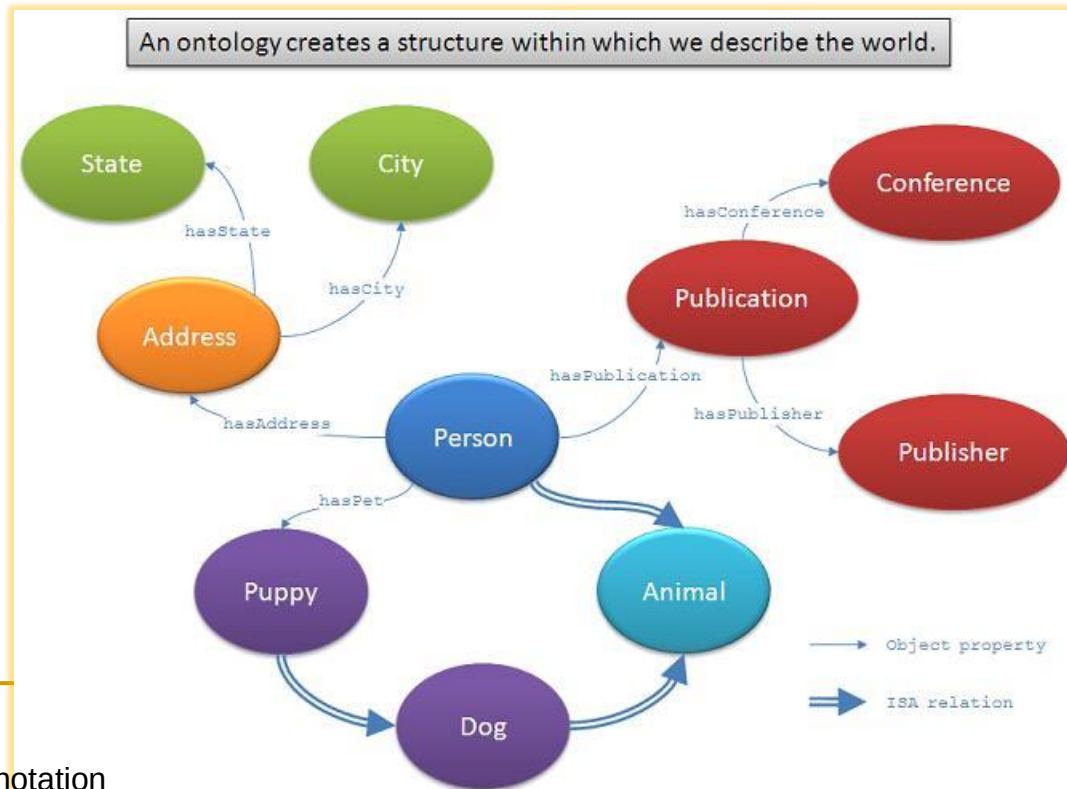
Mạng ngữ nghĩa: Nhược điểm

- Không tồn tại cách diễn dịch (interpretation) chung (chuẩn) – Ngữ nghĩa của các mạng ngữ nghĩa không được định nghĩa một cách chuẩn tắc
- Gặp vấn đề trong việc biểu diễn các thông tin phủ định (negation) và tuyển (disjunction)
 - Vd: “John **does not** go fishing”, “John eats pizza **or** fish and chips”
- Khó khăn trong việc chọn các thành phần cơ bản (primitives) phù hợp
- Khả năng suy diễn hạn chế
- Tốn kém (công sức, thời gian, tiền bạc) khi xây dựng

Ontology

- Một **ontology** là một đặc tả (biểu diễn) **hình thức và rõ ràng** về các **khái niệm**
 - Một **ontology** là một **từ vựng dùng chung**, được dùng để biểu diễn (mô hình) một lĩnh vực cụ thể
 - ❑ Các đối tượng và/hoặc các khái niệm
 - ❑ Các thuộc tính và các quan hệ của chúng
- 
- ```
graph LR; City((City)) -- hasState --> State((State)); Conf((Conf));
```

## ứng dụng cụ thể



# Ontology (2)

- Một ontology có thể được xem như là một cơ sở tri thức
- Một ontology có thể phục vụ các mục đích khác nhau
  - Ví dụ: Mô hình cơ sở dữ liệu (database schema) là một ontology
- Cơ sở tri thức chứa đựng các tri thức cụ thể cần thiết cho việc giải quyết vấn đề của một lĩnh vực

# Ontology: Các động cơ thúc đẩy

- Các động cơ thúc đẩy về mặt công nghệ
  - Rất nhiều các hệ thống dựa trên tri thức sử dụng một ontology mô tả các tri thức của lĩnh vực liên quan
  - Việc xây dựng ontology là một nhiệm vụ rất tốn kém (về thời gian và công sức) trong quá trình phát triển một hệ thống dựa trên tri thức
  - Tại sao không giảm chi phí bằng cách chia sẻ các ontology?
    - Ví dụ: Các ontology “cơ bản” mô tả không gian, thời gian, số lượng, ...
- Động cơ thúc đẩy về mặt khoa học
  - Để hiểu được các vấn đề nền tảng trong quá trình khái niệm hóa (nhận thức các khái niệm) của con người

# Các khía cạnh của một ontology (1)

## ■ Nội dung của ontology

- Các kiểu đối tượng, các kiểu quan hệ
- Ví dụ: Bài toán sắp xếp các khối (blocks)
  - Các lớp đối tượng: Blocks, Robot Hands
  - Các thuộc tính: shapes of blocks, color of blocks
  - Các quan hệ: On, Above, Below, Grasp
  - Các quá trình: thiết kế hoặc xây nên một tòa tháp

## ■ Kiểu của ontology

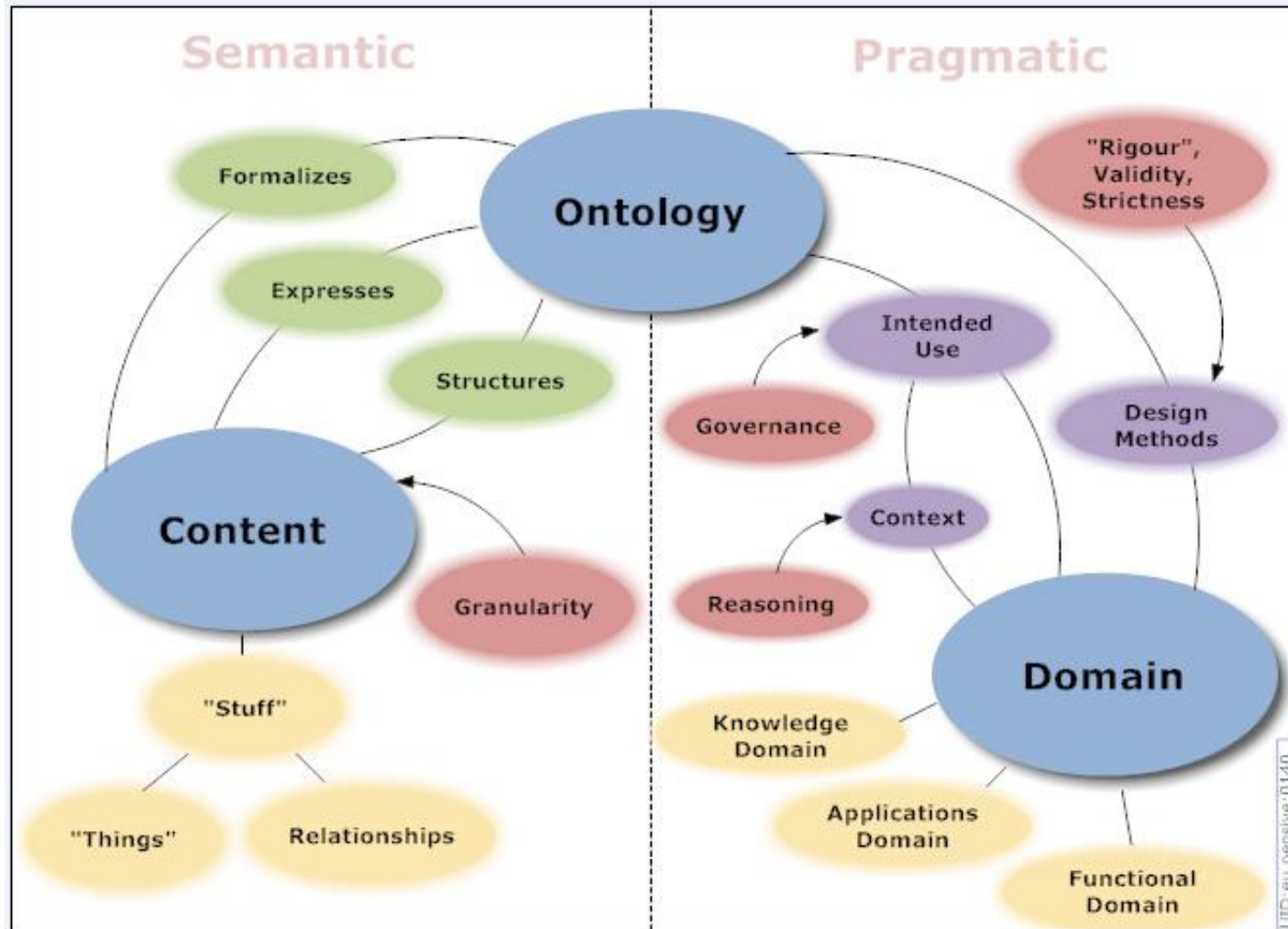
- Các quan hệ phân loại cơ bản (vd: instance-of, subclass) là gì?
- Các định nghĩa của các khái niệm (và các ràng buộc đối với chúng)?
- Khả năng diễn đạt của ngôn ngữ định nghĩa các khái niệm?
- Hướng quá trình (process-centric) hay hướng đối tượng (object-centric)?

# Các khía cạnh của một ontology (2)

- Mục đích sử dụng của ontology
  - Chia sẻ tri thức
    - Ví dụ: Giữa những người sử dụng, giữa các hệ thống, ...
  - Sử dụng lại tri thức
    - Ví dụ: Sử dụng lại (một phần) tri thức khi các mô hình hoặc hệ thống thay đổi
- Việc xây dựng ontology
  - Ontology sẽ được thu thập hay xây dựng?
  - Nếu được thu thập, thì cần kiểm tra:
    - Chất lượng của tri thức?
    - Sự khác biệt về nội dung?
    - Sự tin cậy của tri thức?
    - Các khả năng sử dụng trong tương lai?

# Ontology

(Lý  
thuyết  
cần  
xem  
xét)



(Thực  
tế cần  
xem  
xét)

Source: [http://www.emiliosanfilippo.it/?page\\_id=1172](http://www.emiliosanfilippo.it/?page_id=1172)

# Xây dựng ontology: các vấn đề

- Xác định giới hạn (phạm vi) và mục đích sử dụng
- Cân nhắc việc sử dụng lại các ontology đã có và liên quan đến lĩnh vực đang xét
- Liệt kê các khái niệm
- Định nghĩa sự phân loại
- Định nghĩa các thuộc tính
- Định nghĩa các khía cạnh (các ràng buộc)
- Xác định các ví dụ cụ thể (instances) ≠ *mạng ngữ nghĩa*
  - Kiểm tra các bất thường (anomalies)

*mô tả rất chi tiết  
k°p số lượng như trong mạng ngữ nghĩa*

# Phạm vi và mục đích sử dụng

- Không tồn tại một ontology “chuẩn” cho một lĩnh vực cụ thể
  - Một ontology là một sự khái quát hóa về một lĩnh vực cụ thể, và luôn tồn tại nhiều ontologies phù hợp
- Việc khái quát hóa này nên tính đến: mức k/quát hay cụ thể? khả năng, tiềm lực
  - Việc sử dụng trong tương lai của ontology (mục đích sử dụng)
  - Các khả năng mở rộng (của ontology) có thể dự đoán trước
- Các câu hỏi cần phải được trả lời ở giai đoạn này:
  - Ontology này được dùng cho lĩnh vực nào?
  - Ontology này sẽ được dùng để làm gì?
  - Những kiểu câu hỏi nào mà ontology này có thể đưa ra câu trả lời?
  - Ai sẽ dùng và bảo trì ontology này?

# Cần nhắc sử dụng lại các ontology

- Với sự phát triển rất nhanh chóng của Internet và của lĩnh vực Semantic Web, sẽ tồn tại (có) rất nhiều ontologies có thể khai thác

# Liệt kê các khái niệm

- Xác định (theo một danh sách) tất cả các khái niệm liên quan sẽ xuất hiện trong ontology
  - Các danh từ (nouns) thường là cơ sở để xác định các tên lớp (class names)
  - Các động từ (verbs) hoặc cụm động từ (verb phrases) thường là cơ sở để xác định các tên thuộc tính (property names)
- Nên sử dụng các công cụ xây dựng tri thức, để thu được
  - Tập các khái niệm
  - Cấu trúc (phân cấp) ban đầu của các khái niệm này

# Định nghĩa sự phân loại

- Các khái niệm liên quan cần được tổ chức lại với nhau thành một cấu trúc phân cấp phân loại (a taxonomic hierarchy)
  - Hai chiến lược thường được dùng: top-down vs. bottom-up
  - Cần cân nhắc đến hai yếu tố: sự tin cậy (chính xác) và hiệu quả (suy diễn)
- Đảm bảo rằng phân cấp thu được thực sự là một phân loại khái niệm
  - Nếu A là một lớp con của B, thì mọi ví dụ của A cũng phải là ví dụ của B

# Định nghĩa các thuộc tính

- Thường được thực hiện kết hợp với bước trước (định nghĩa sự phân loại)
- Yêu cầu: Nếu  $A$  là một lớp con của  $B$ , thì mọi thuộc tính của  $B$  đều phải được áp dụng cho  $A$ 
  - Nên gắn các thuộc tính với lớp cao nhất (phù hợp) trong cấu trúc phân cấp
- Khi xác định một thuộc tính cho một lớp, cần ngay lập tức xác định các thông tin về miền (data type) và giá trị (value domain) của thuộc tính đó

# Định nghĩa các ràng buộc

- Các ràng buộc về số chiều (số phần tử của một tập hợp)
- Các giá trị có thể được gán cho
- Các đặc điểm của quan hệ
  - Tính đối xứng, bắc cầu, đảo, ...

| Template Slots |          |             |                                           |  |  |  |  |  |  |
|----------------|----------|-------------|-------------------------------------------|--|--|--|--|--|--|
| Name           | Type     | Cardinality | Other Facets                              |  |  |  |  |  |  |
| body           | Symbol   | single      | allowed-values={FULL,MEDIUM,LIGHT}        |  |  |  |  |  |  |
| color          | Symbol   | single      | allowed-values={RED,ROSÉ,WHITE}           |  |  |  |  |  |  |
| flavor         | Symbol   | single      | allowed-values={DELICATE,MODERATE,STRONG} |  |  |  |  |  |  |
| grape          | Instance | multiple    | classes={Wine grape}                      |  |  |  |  |  |  |
| maker          | Instance | single      | classes={Winery}                          |  |  |  |  |  |  |
| name           | String   | single      |                                           |  |  |  |  |  |  |
| sugar          | Symbol   | single      | allowed-values={DRY,SWEET,OFF-DRY}        |  |  |  |  |  |  |

([protege.stanford.edu/amia2003/AMIA2003Tutorial.ppt](http://protege.stanford.edu/amia2003/AMIA2003Tutorial.ppt))

# Xác định các ví dụ

- Bổ sung vào ontology các ví dụ (instances) cụ thể
- Số lượng các ví dụ thường lớn hơn (nhiều) số lượng các lớp (classes)
- Việc bổ sung các ví dụ vào một ontology thường được làm thủ công (manually)
  - Lấy từ các nguồn dữ liệu sẵn có (vd: từ các cơ sở dữ liệu)
  - Trích tự động từ một tập các dữ liệu văn bản

# Kiểm tra các bất thường

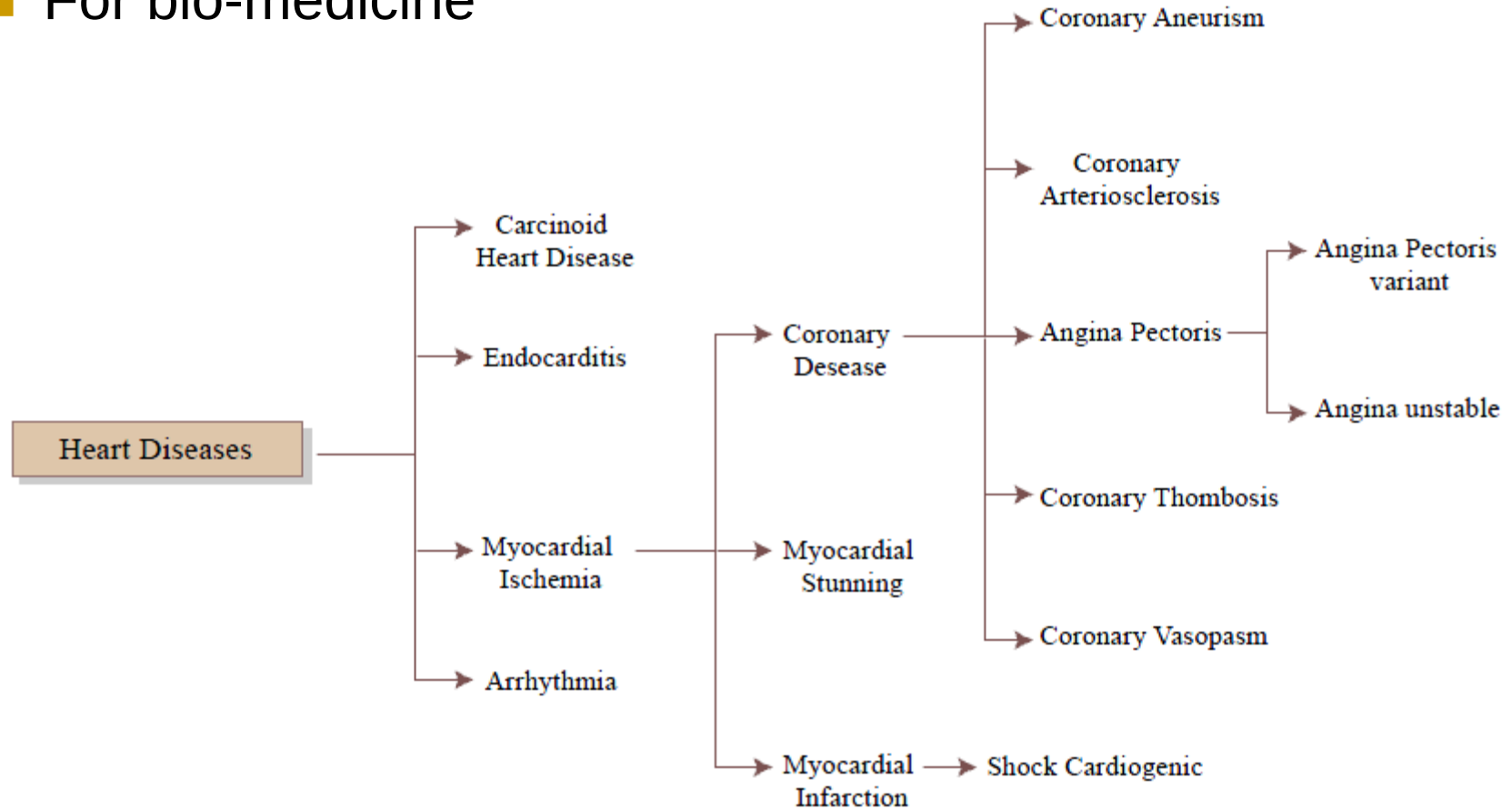
- Để kiểm tra các mâu thuẫn có thể có trong một ontology, (hoặc có trong một ontology đi kèm với các ví dụ)
- Các ví dụ của các mâu thuẫn
  - Không tương thích về miền, về giá trị, về các đặc tính bắc cầu, đối xứng, đảo,...
  - Không tương thích với ràng buộc về số chiều của một thuộc tính

# Các ví dụ về ontology

- CYC (dùng chung, tổng quát)
  - $10^5$  kiểu khái niệm,  $10^6$  tiên đề
- SENSUS (mở rộng của WordNet, dùng cho xử lý văn bản)
  - 70.000 khái niệm
- SUMO (tổng quát)
  - 1000 khái niệm, 4200 đánh giá
- UMLS (lĩnh vực y-sinh)
  - 135 kiểu ngữ nghĩa, 54 quan hệ ngữ nghĩa, 975.354 khái niệm
- WordNet (từ vựng)
  - 152.059 dạng từ, 115,424 cụm từ

# UMLS ontology

## ■ For bio-medicine



(Figure by MIT OCW)

# CYC ontology

- Encode all of human common sense knowledge

